Accenture

TASK 2

AI-Assisted Deployment & Release Operations

Continuation of Task 1 Research Analysis

Gaurav Budhwani
Harsh Sandeep Patil

TASK 1 FINDINGS:

Rank	Platform	Strength	Pricing
#1	**Harness**	Native AI Continuous Verification & Rollbacks	**$50-$150/dev/month**
#2	**Dynatrace Davis AI**	Causal AI Root Cause Analysis; 274% ROI	**Consumption-based**
#3	**Keptn**	SLI/SLO Quality Gates; Kubernetes-native	**Open Source (Free)**
#4	**GitHub Copilot**	AI-assisted YAML & artifact generation	**Free tier available**
#5	**Argo CD**	GitOps declarative state management	**Open Source (Free)**
#6	**GitLab Duo**	Context-aware DevSecOps agents	**$29-$99/user/month**
#7	**Jenkins X**	Predictive analytics from builds	**Open Source (Immature)**

AI Tools : Selection & Reasons

Financial Barriers

Most of the deployment and release platforms offer only paid enterprise-tier plans
Example: Dynatrace, Gitlab-Duo, etc.

Limited Free Tiers

Most platforms which do have free-tier plans strip out AI-specific capabilities as a premium feature.
Example: Harness, CircleCI, etc.

Security Concerns

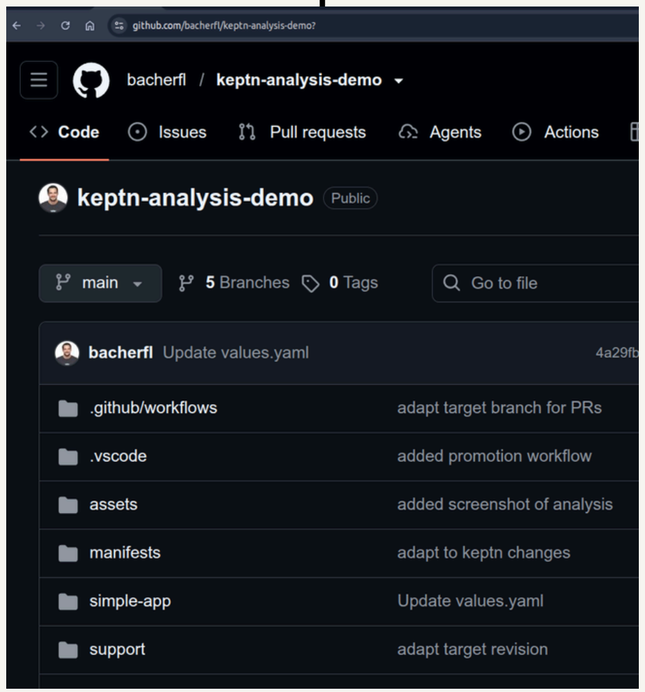
The potential of credential leakage exists on non-reputable platforms that provide free AI services with automation promises (e.g. Github PATs, API keys etc.) .

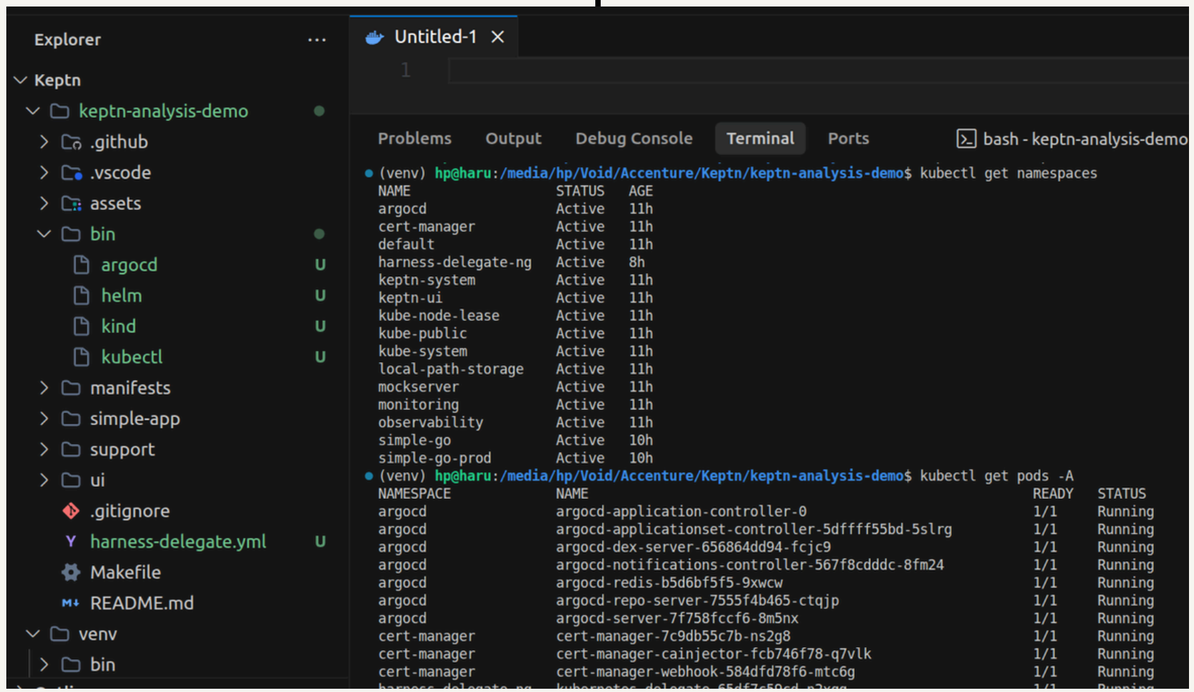
These are essential for automation but it also create a risk of leakage.

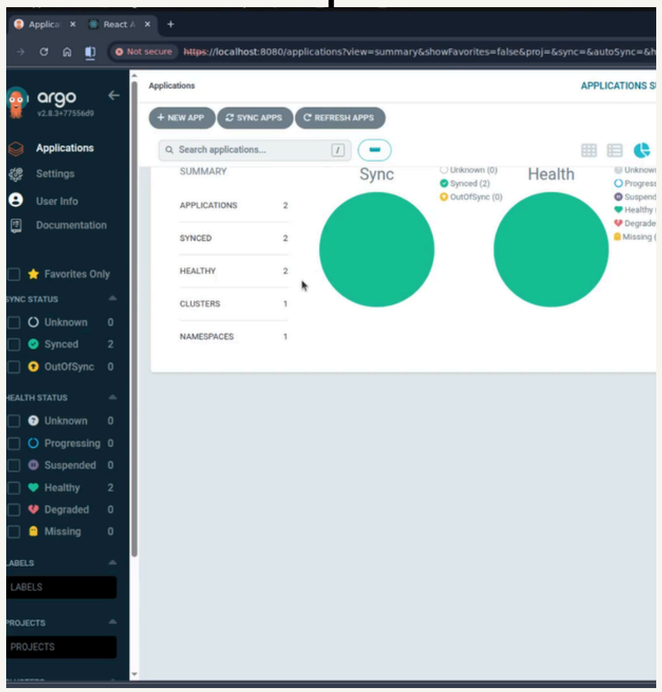
DEMO 1: Keptn & Argo CD

Demo Repository (Prerequisite + setup) → **Deploy to** (Kubernetes Local) → **Argo CD** (Continuous sync) → **KEPTN** Quality Gate → **Pass / Fail** Decision → **STATUS UPDATE**

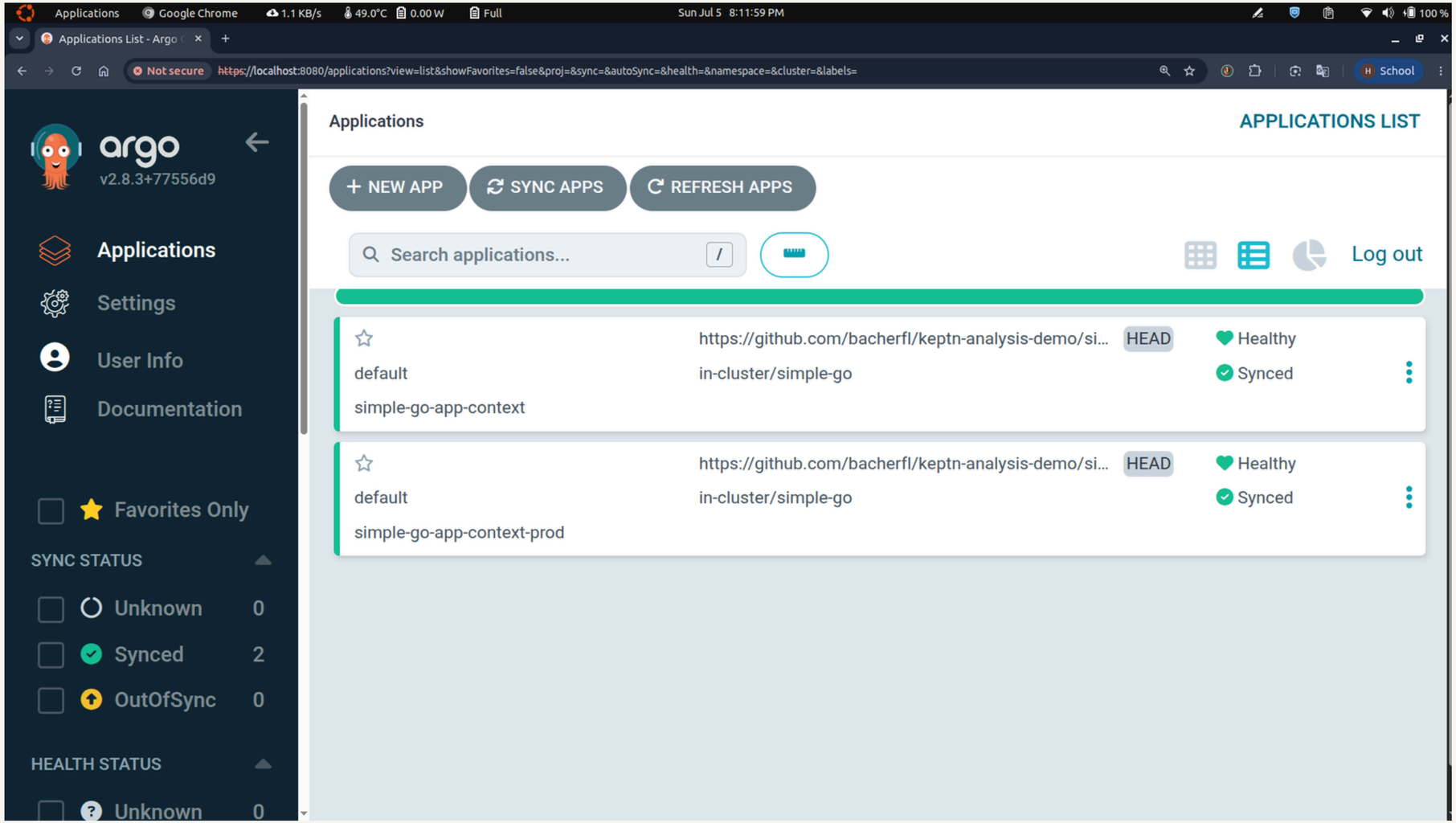
NEXT SLIDE







ArgoCD Web UI



What is this image ?

ArgoCD dashboard showing the application deployments and their healthy/synced statuses.

What does this mean?

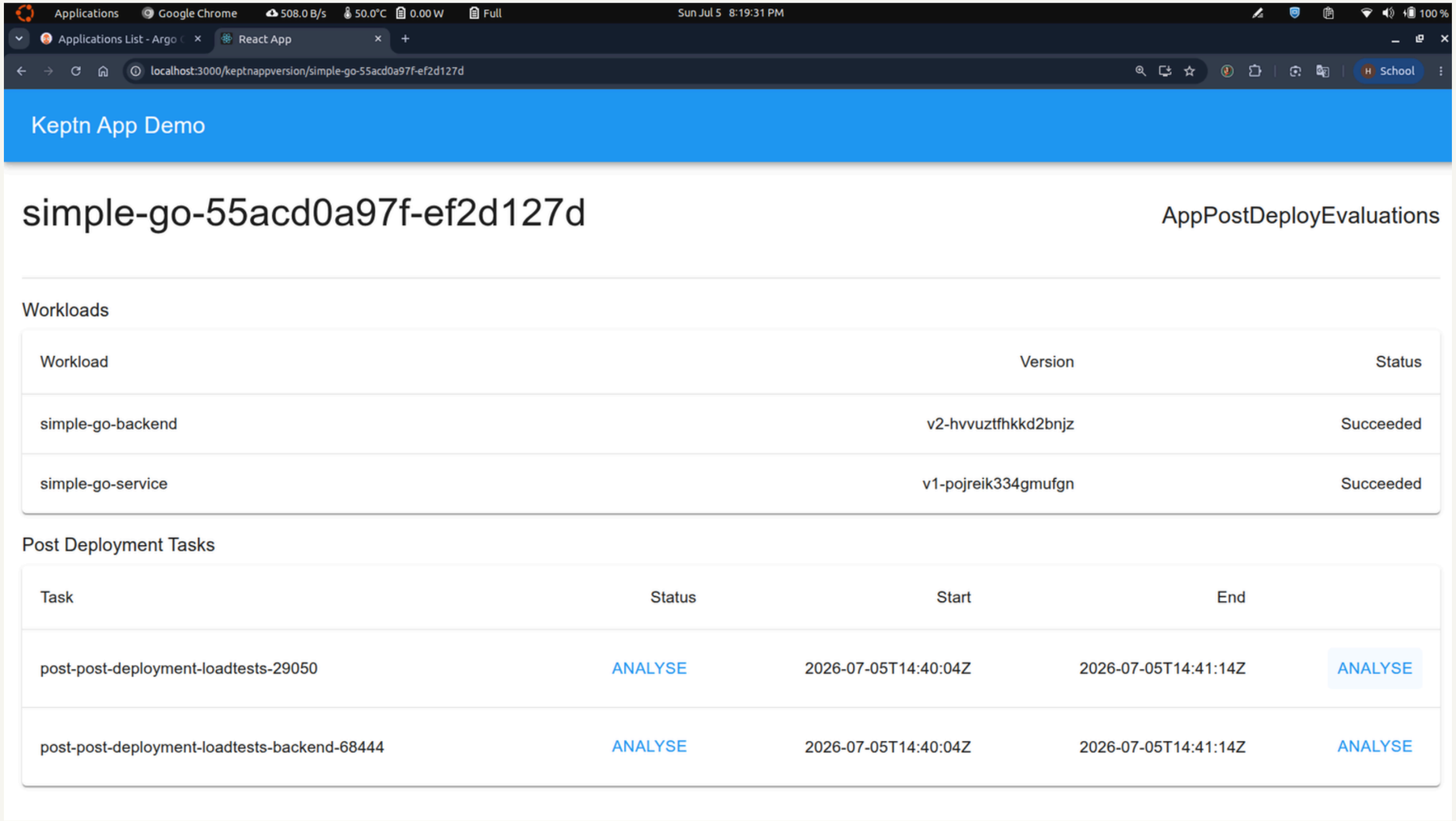
Declarative Continuous Delivery (GitOps) in action. ArgoCD acts as a continuous reconciler, reading the application blueprints directly from a Git repository and ensuring the state of our local kind cluster matches the repository perfectly.

Why do we have to do this?

- **Automated Sync Engine**
- **Visual Resource Mapping**
- **Zero Drift**

Instead of manually deploying apps using command scripts. ArgoCD is watching our repository. The moment code is pushed, ArgoCD automatically handles the SDLC inside our cluster without exposing cluster secrets to outside platforms

Keptn Web UI



What is this image ?

Keptn dashboard showing the automatic post-deployment evaluations and their statuses

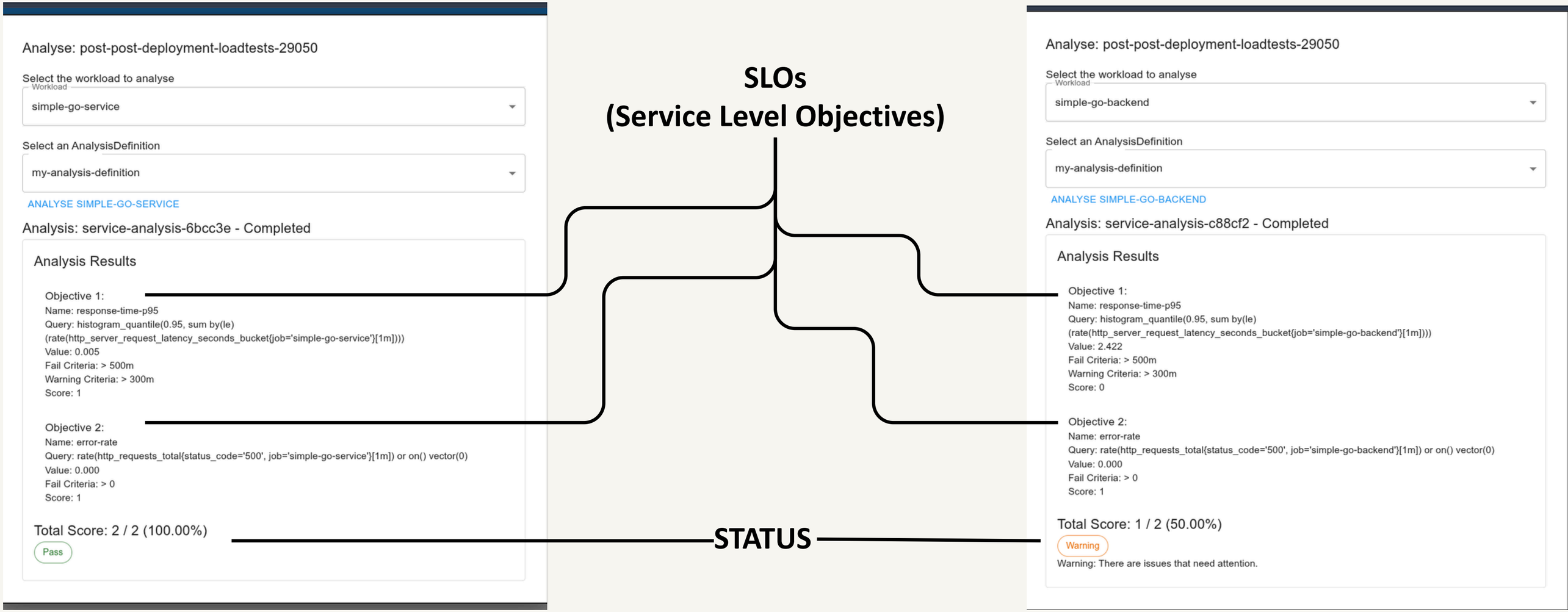
What does this mean?

This shows automated Site Reliability Engineering (SRE) evaluation. Keptn doesn't just check if a container is running; it runs load tests and pulls live metrics from Prometheus to determine if the application is safe or broken.

Why do we have to do this?

Because "running" does not mean "working." Just because a container successfully starts up doesn't mean it can handle real user traffic or that it won't crash under pressure. Automating this process catches hidden bugs, performance bottlenecks, and memory leaks immediately after a deployment, allowing us to catch issues before our users do.

Site Reliability Engineering (SRE) via Keptn



DEMO 2: Github Copilot

Setup & Approach

Setup Steps

1. Created a folder flask-copilot-demo, open in VS Code
2. Install GitHub Copilot extension from marketplace
3. Sign in to GitHub account, verify Chat is working
4. Switch to Agent Mode (not Ask mode) — critical for file creation

Stack: Flask + Copilot + GitHub Actions + Docker + K8s + Helm

The ONLY Manually Created File — app.py

```python
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return {
        "status": "healthy",
        "message": "Deployment Successful"
    }

if __name__ == "__main__":
    app.run(debug=True)
```

Key Point:
This is a minimal 15-line Flask app. It has NO Dockerfile, NO CI/CD pipeline, NO Kubernetes manifests, NO deployment strategy, NO health checks, NO incident playbooks. Everything else was generated entirely by GitHub Copilot through natural language prompts.

Generate Production Deployment Artifacts

PROMPT

```
"You are a Senior DevOps Engineer. I have a simple Flask application. Your task is to prepare this application for production deployment. Create every required deployment artifact. Requirements: 1. Generate requirements.txt. 2. Generate a production-ready Dockerfile. 3. Explain every line of the Dockerfile. 4. Create a .dockerignore. 5. Explain why each file is required during SDLC Phase 4. Create all files automatically inside the project."
```

requirements.txt

Python Dependencies

- Flask with pinned version
- Gunicorn (production WSGI server)
- Reproducible dependency resolution

Dockerfile

Production Container Image

- Python 3.11-slim base image
- Non-root user for security
- Gunicorn entrypoint, port 5000
- Optimized dependency caching

.dockerignore

Build Context Optimization

- Excludes .git, __pycache__, .env
- Excludes venv, *.pyc files
- Reduces image size & prevents leaks

Phase 4 Relevance: These are the foundational deployment artifacts — without them, the app cannot be containerized or deployed to any orchestration platform.

Generate Kubernetes Deployment Manifests

PROMPT

```
"Generate Kubernetes deployment files. Requirements: Deployment, Service, ConfigMap, Ingress. Use Rolling Update strategy. Use readiness probes. Use liveness probes. Explain every YAML section. Comment every field."
```

deployment.yaml

Deployment

- 3 replicas for high availability
- Rolling Update: maxUnavailable 1, maxSurge 1
- Readiness probe: HTTP GET / every 10s
- Liveness probe: HTTP GET / every 15s
- Resource limits: 250m CPU, 256Mi memory

service.yaml

Service

- ClusterIP type for internal routing
- Port 80 mapped to container port 5000
- Selector matching deployment labels
- Enables load balancing across pods

configmap.yaml

ConfigMap

- FLASK_ENV: production
- LOG_LEVEL: info
- APP_NAME: flask-app
- Externalized from application code

ingress.yaml

Ingress

- Host-based routing rules
- Path: / routing to backend service
- Port 80 for external traffic
- Ready for TLS termination

Rolling Update strategy ensures zero-downtime deployments. Probes ensure K8s only sends traffic to healthy, ready pods.

Helm Chart, Production Practices & Architecture

Prompt 4: Helm Chart Generation

helm/Chart.yaml

Name, version 0.1.0, appVersion 1.0.0

helm/values.yaml

Parameterized: image repo, tag, replicas, limits, ingress

helm/templates/

Go-templated K8s manifests with {{ .Values.* }}

Why Helm: Parameterized deployments across dev/staging/prod. Versioned releases. helm rollback for instant revert.

Prompt 5: Production Best Practices

Blue-Green

Two identical envs. Traffic switches atomically. Instant rollback.

Canary

Gradual shift: 10% → 25% → 50% → 100%. Monitor at each stage.

Health Checks

HTTP probes with configurable initialDelay, period, threshold.

Auto Rollback

If health checks fail during canary, auto-revert to previous.

Prompt 6: Architecture Diagram

Summary of Prompts 4-6:

Prompt 4 gives us parameterized, repeatable deployments via Helm. Prompt 5 adds zero-downtime strategies (Blue-Green, Canary) with automatic rollback.
Prompt 6 generates an end-to-end architecture visualization showing the complete path from developer workstation to production.

Thank you

AI Acting as a Deployment Review Engineer

PROMPT

```
"You are acting as an AI Deployment Assistant. Review all deployment files. Find possible mistakes. Suggest improvements. Explain: Security issues, Performance improvements, Deployment risks, Scaling improvements, Rollback strategy. Output the report like an AI DevOps Engineer reviewing production code."
```

Security Issues

- Running as root — add USER directive
- No network policies defined
- Secrets in plaintext — use K8s Secrets/Vault
- No container image scanning in pipeline
- Missing securityContext: runAsNonRoot

Performance

- No HPA (Horizontal Pod Autoscaler)
- Resource limits may be too conservative
- No caching layer (Redis/Memcached)
- Gunicorn workers should match CPU cores

Deployment Risks

- No Pod Disruption Budget configured
- All pods could be evicted during maintenance
- Single namespace (no env isolation)
- No resource quotas at namespace level

Scaling

- Add HPA: min 2, max 10, target CPU 70%
- Implement cluster autoscaler
- Consider Vertical Pod Autoscaler
- Right-size resource requests/limits

Rollback Strategy

- kubectl rollout undo deployment/flask-app
- Maintain last 3 revision history
- Implement Helm rollback for charts
- Add pre-rollback health snapshot

This demonstrates AI acting as a Senior DevOps Engineer — reviewing infrastructure code and providing actionable feedback in seconds.

AI Diagnosing Deployment Failures

PROMPT

```
"Imagine the deployment failed. Logs: ImagePullBackOff, CrashLoopBackOff, Container exited with code 1. Act as an AI Deployment Engineer.
Identify the root cause. Suggest fixes. Suggest rollback procedure. Generate an incident report."
```

ERROR	ROOT CAUSE IDENTIFIED BY AI	FIX RECOMMENDED BY AI
ImagePullBackOff	Docker image name/tag incorrect or doesn't exist in registry. OR registry credentials (imagePullSecrets) missing/expired.	Verify image: docker pull <image>. Check imagePullSecrets in deployment. Validate Docker Hub credentials in K8s secrets.
CrashLoopBackOff	Application crashes on start — missing env variables, incorrect CMD in Dockerfile, or Python import errors.	Check logs: kubectl logs <pod>. Verify all env vars in ConfigMap. Test container locally: docker run <image>.
Exit Code 1	Unhandled Python exception during Flask startup — missing dependency, syntax error, or config issue.	Read full stack trace from pod logs. Verify requirements.txt is complete. Test app locally before deploying.

Rollback Procedure Generated by AI

1. kubectl rollout undo deployment/flask-app
2. kubectl rollout status deployment/flask-app
3. kubectl get pods – verify all running
4. Verify application endpoints responding

Incident Report Structure Generated

Incident ID, Severity, Timestamp
Summary of what happened
Root cause analysis with evidence
Actions taken + preventive measures

Before vs. After: The AI Transformation

BEFORE | 1 File | Manual

flask-copilot-demo/
app.py (15 lines of code)
No Dockerfile
No CI/CD pipeline
No Kubernetes manifests
No Helm charts
No deployment strategy
No health checks or probes
No incident playbooks
No deployment checklist

Estimated: 2-4 days of manual effort

10 AI Prompts

AFTER | 15+ Files | AI-Generated

app.py
(original)
requirements.txt
Python deps
Dockerfile
Container image
.dockerignore
Build optimization
deploy.yml
CI/CD pipeline
deployment.yaml
K8s Deployment
service.yaml
K8s Service
configmap.yaml
K8s Config
ingress.yaml
K8s Ingress
helm/ (3 files)
Helm chart
deployment-review.md
AI security analysis
incident-report.md
AI failure diagnosis
deployment-checklist.md
Pre-prod validation